On the Impossibility of Autonomous Self-Improvement to General Intelligence in Stochastic Language Models: A Formal Proof via Information Theory, Gödelian Limits, and Distributional Collapse

Revised Edition with Counterargument Analysis

Filip Pajalic

2025

Abstract

We prove that large language models (LLMs) based on stochastic next-token prediction cannot, through autonomous self-improvement without external human influence, converge to artificial general intelligence (AGI). We formalize the LLM as a conditional probability estimator trained via maximum likelihood on a fixed distributional support and establish independent impossibility results: (1) the information-theoretic ceiling imposed by training data entropy; (2) distributional collapse under recursive self-training; (3) Gödelian incompleteness bounds on self-verification, with an analysis of why these bounds apply asymmetrically to computational systems but not to quantum-biological systems such as the human brain; (4) the stochastic gradient descent fixed-point trap; (5) the fundamental gap between statistical pattern completion and general reasoning; (6) the computational complexity barrier, showing that self-improvement requires solving NP-hard optimization at each step, and that LLMs' own failure on NP-hard problems despite massive statistical exposure constitutes empirical evidence for $P \neq NP$, which strengthens the impossibility; (7) the context-computation paradox, showing that self-correction requires resources that grow without bound relative to the system's finite processing capacity; and (8) error compounding under recursive self-modification. We address counterarguments regarding inference-time search, reinforcement learning with self-play, tool-augmented systems, and multi-agent architectures, showing that each either falls within the proof's scope or constitutes external human influence that confirms rather than refutes the central claim. We further provide a control theory perspective showing that LLMs violate the fundamental prerequisites for self-regulation, and present empirical corroboration from the SWE-CI benchmark, where 18 models across 100 long-term code maintenance tasks exhibit the predicted error divergence and regression patterns. The conjunction of these results establishes that autonomous AGI emergence from the stochastic language modeling framework is not an engineering challenge but a mathematical impossibility.

Contents

1 Scope and Definitions **3**

1.1 The Large Language Model as a Stochastic System 3

1.2 Defining Autonomous Self-Improvement 4

2 Information-Theoretic Ceiling **4**

3 Distributional Collapse Under Self-Training **5**

4 Gödelian Limits on Self-Verification **6**
4.1 Why Gödel Does Not Apply Symmetrically to Humans 7

5 The SGD Fixed-Point Trap **7**

6 The Pattern Completion Barrier **8**

7 The Computational Complexity Barrier (P $\neq$ NP) **9**

8 The Context-Computation Paradox **10**

9 Error Compounding in Recursive Self-Modification **13**

10 Main Theorem **14**

11 Discussion **15**
11.1 What the Proof Shows 15
11.2 What Would Escape These Bounds 15
11.3 The Stochastic Parrot, Formalized 15
11.4 Human Influence Is Provably Necessary 15
11.5 A Control Theory Perspective 15
11.6 The Multi-Agent Correlation Problem 16
11.7 Why Noise Cannot Substitute for Human Influence 17
11.8 Empirical Corroboration: The SWE-CI Benchmark 17
11.8.1 Experimental Setup 17
11.8.2 Key Result: Zero-Regression Rate 17
11.8.3 Mapping to Theoretical Results 18
11.8.4 Within-Run Degradation vs. Across-Release Improvement 18

1 Scope and Definitions

We begin by precisely delineating what this proof does and does not claim.

Remark 1.1 (Scope)**.** *This proof addresses the following specific question:* Can a system based on autoregressive stochastic language modeling, trained via maximum likelihood estimation (or equivalent objectives), autonomously self-improve to AGI without any form of human influence at any stage of the self-improvement process?

*We do **not** claim:*

- *That AGI is impossible by any means whatsoever.*
- *That hybrid systems combining LLMs with other components cannot advance.*
- *That human-guided improvement of LLMs is bounded.*

*We **do** claim: within the stochastic autoregressive framework operating autonomously, the mathematical structure of the system provably prevents convergence to AGI. Any system that escapes these bounds does so by incorporating elements that constitute human influence or that depart from the stochastic language modeling paradigm—thereby confirming rather than refuting our thesis.*

1.1 The Large Language Model as a Stochastic System

Definition 1.2 (Autoregressive Language Model)**.** *An autoregressive language model is a parameterized conditional probability distribution over a vocabulary $\mathcal{V} = \{v_1, v_2, \ldots, v_V\}$:*

$$p_{\boldsymbol{\theta}}(x_t \mid x_{<t}) = \mathrm{softmax}(\boldsymbol{W}_{\mathrm{out}} \cdot h_L(\boldsymbol{x}_{<t}; \boldsymbol{\theta}))_{x_t} \tag{1}$$

where $\boldsymbol{\theta} \in \mathbb{R}^p$ is the parameter vector (with p on the order of 10^9 to 10^{12}), and h_L is the output of an L-layer transformer network. The full sequence probability factorizes as:

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(x_t \mid x_{<t}) \tag{2}$$

Definition 1.3 (Transformer Computation)**.** *Each transformer layer $\ell \in \{1, \ldots, L\}$ computes:*

$$\boldsymbol{z}^{(\ell)} = \mathrm{LayerNorm}\Big(\boldsymbol{h}^{(\ell-1)} + \mathrm{MultiHeadAttn}(\boldsymbol{h}^{(\ell-1)})\Big) \tag{3}$$

$$\boldsymbol{h}^{(\ell)} = \mathrm{LayerNorm}\Big(\boldsymbol{z}^{(\ell)} + \mathrm{FFN}(\boldsymbol{z}^{(\ell)})\Big) \tag{4}$$

where multi-head attention with H heads is:

$$\mathrm{MultiHeadAttn}(\boldsymbol{h}) = \boldsymbol{W}_O \left[\bigoplus_{i=1}^{H} \mathrm{softmax}\left(\frac{\boldsymbol{Q}_i \boldsymbol{K}_i^\top}{\sqrt{d_k}} \right) \boldsymbol{V}_i \right] \tag{5}$$

with $\boldsymbol{Q}_i = \boldsymbol{W}_Q^{(i)} \boldsymbol{h}$, $\boldsymbol{K}_i = \boldsymbol{W}_K^{(i)} \boldsymbol{h}$, $\boldsymbol{V}_i = \boldsymbol{W}_V^{(i)} \boldsymbol{h}$, and:

$$\mathrm{FFN}(\boldsymbol{z}) = \boldsymbol{W}_2 \cdot \sigma(\boldsymbol{W}_1 \boldsymbol{z} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \tag{6}$$

for activation function σ (typically GELU or SiLU).

Definition 1.4 (Training Objective)**.** *The model is trained to minimize the cross-entropy loss:*

$$\mathcal{L}(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{x} \sim \mathcal{D}_{\mathrm{train}}} \left[\frac{1}{T} \sum_{t=1}^{T} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}) \right] = H(p_{\mathrm{data}}) + \mathrm{KL}(p_{\mathrm{data}} \| p_{\boldsymbol{\theta}}) \tag{7}$$

1.2 Defining Autonomous Self-Improvement

Definition 1.5 (Human Influence). *We define* human influence *as any of the following introduced at step $k \geq 1$:*

1. *New human-curated or human-generated training data,*
2. *Human-designed reward functions, preference labels, or evaluation criteria,*
3. *Human-designed filtering, selection, or ranking mechanisms applied to model outputs,*
4. *Human-designed tools, environments, or external verifiers (including game rules, physics engines, code interpreters, proof checkers),*
5. *Human-designed architectural modifications or hyperparameter schedules.*

Note that items (2)–(4) include systems where humans designed the evaluation mechanism before *the self-improvement loop begins. A reward function crafted by humans is human influence even if no human is present during training. This is the critical distinction: the intelligence embedded in the design of the feedback signal originates from human cognition.*

Definition 1.6 (Autonomous Self-Improvement). *An autonomous self-improvement process is a sequence of models $\{\boldsymbol{\theta}_k\}_{k=0}^{\infty}$ where:*

$$\boldsymbol{\theta}_{k+1} = \mathcal{A}\Big(\boldsymbol{\theta}_k, \mathcal{D}_k^{\text{self}}\Big) \tag{8}$$

where $\mathcal{A}$ is a training algorithm and $\mathcal{D}_k^{\text{self}}$ is data generated by the model:

$$\mathcal{D}_k^{\text{self}} = \Big\{\boldsymbol{x}^{(i)} \sim p_{\boldsymbol{\theta}_k}(\cdot)\Big\}_{i=1}^{N_k} \tag{9}$$

with ***no human influence*** *(Definition 1.5) at any step $k \geq 1$.*

Definition 1.7 (Artificial General Intelligence — Operational). *A system achieves AGI if there exists a model $\boldsymbol{\theta}^*$ such that for any computable function $f : \Sigma^* \to \Sigma^*$ and any $\epsilon > 0$:*

$$\Pr_{\boldsymbol{x} \sim \mathcal{D}_f}\left[p_{\boldsymbol{\theta}^*}(\cdot \mid \boldsymbol{x}) \text{ assigns highest probability to } f(\boldsymbol{x})\right] \geq 1 - \epsilon \tag{10}$$

2 Information-Theoretic Ceiling

Theorem 2.1 (Self-Generated Data Cannot Increase Information). *Let p_{true} denote the ground-truth distribution over all valid reasoning chains. For the autonomous self-improvement sequence $\{\boldsymbol{\theta}_k\}$:*

$$I(p_{\boldsymbol{\theta}_k}; p_{\text{true}}) \leq I(p_{\boldsymbol{\theta}_0}; p_{\text{true}}) \leq I(\mathcal{D}_{\text{train}}; p_{\text{true}}) \tag{11}$$

Proof. The self-generated dataset $\mathcal{D}_k^{\text{self}}$ is produced by sampling from $p_{\boldsymbol{\theta}_k}$. By the data processing inequality [2], for the Markov chain:

$$p_{\text{true}} \to \mathcal{D}_{\text{train}} \to \boldsymbol{\theta}_0 \to p_{\boldsymbol{\theta}_0} \to \mathcal{D}_0^{\text{self}} \to \boldsymbol{\theta}_1 \to \cdots \to \boldsymbol{\theta}_k \tag{12}$$

each transformation satisfies:

$$I(\boldsymbol{\theta}_{k+1}; p_{\text{true}}) \leq I(\boldsymbol{\theta}_k; p_{\text{true}}) \tag{13}$$

with strict inequality whenever $p_{\boldsymbol{\theta}_k} \neq p_{\text{true}}$. □

Objection 1 (Inference-Time Search and Latent Information). *Chain-of-thought reasoning, beam search, and tree search can extract capabilities "latent" in the weights without adding new data. Doesn't this violate the information ceiling?*

Response 1. *No. Inference-time search reorganizes and better utilizes information already present in $\boldsymbol{\theta}_k$. This is analogous to lossless compression: you can rearrange bits to make them more useful, but you cannot create bits that were not there. Formally, let $\mathcal{S}$ be any deterministic search procedure applied at inference time. Then:*

$$I(\mathcal{S}(p_{\boldsymbol{\theta}_k}); p_{\text{true}}) \leq I(p_{\boldsymbol{\theta}_k}; p_{\text{true}}) \leq I(\mathcal{D}_{\text{train}}; p_{\text{true}}) \tag{14}$$

by the data processing inequality (since $\mathcal{S}$ is a function of $p_{\boldsymbol{\theta}_k}$). Search can improve utilization *of existing information up to a ceiling, but that ceiling is fixed by the training data. Better utilization is not the same as new knowledge. The model may solve problems it previously failed on by searching more effectively, but it cannot solve problems whose solutions require information absent from the training distribution. This is the distinction between* extraction *(bounded) and* creation *(what AGI requires for novel domains).*

3 Distributional Collapse Under Self-Training

Theorem 3.1 (Entropy Contraction Under Self-Distillation)**.** *Let $p_{\boldsymbol{\theta}_k}$ be the model at step k, and let $p_{\boldsymbol{\theta}_{k+1}}$ be obtained by training on N samples from $p_{\boldsymbol{\theta}_k}$ with no external filtering or selection. Then:*

$$H(p_{\boldsymbol{\theta}_{k+1}}) \leq H(p_{\boldsymbol{\theta}_k}) \tag{15}$$

with strict inequality for finite N. The support contracts monotonically:

$$\text{supp}(p_{\boldsymbol{\theta}_{k+1}}) \subseteq \text{supp}(p_{\boldsymbol{\theta}_k}) \tag{16}$$

Proof. **Step 1: Sampling prunes the tail.** The empirical distribution $\hat{p}_k$ over N samples satisfies: for any sequence with $p_{\boldsymbol{\theta}_k}(\boldsymbol{x}) < 1/N$:

$$\Pr[\boldsymbol{x} \notin \mathcal{D}_k^{\text{self}}] = (1 - p_{\boldsymbol{\theta}_k}(\boldsymbol{x}))^N \geq \left(1 - \frac{1}{N}\right)^N \to e^{-1} \tag{17}$$

Step 2: Training concentrates. Training $\boldsymbol{\theta}_{k+1}$ to minimize $\text{KL}(\hat{p}_k \| p_{\boldsymbol{\theta}})$ drives $p_{\boldsymbol{\theta}_{k+1}}$ toward $\hat{p}_k$, which has smaller support and lower entropy than $p_{\boldsymbol{\theta}_k}$.

Step 3: Induction. $H(p_{\boldsymbol{\theta}_0}) \geq H(p_{\boldsymbol{\theta}_1}) \geq H(p_{\boldsymbol{\theta}_2}) \geq \cdots \geq 0$. The model becomes progressively less diverse. □

Corollary 3.2 (Loss of Generality)**.** *AGI requires* $\text{supp}(p_{\boldsymbol{\theta}^*}) \supseteq \bigcup_{f \in \mathcal{F}} \text{supp}(\mathcal{D}_f)$. *Since self-training contracts the support, the model moves* away *from AGI with each iteration.*

Objection 2 (Filtering and Reinforcement Learning)**.** *AlphaZero improved through self-play. If a model generates 100 solutions and trains only on the best one, entropy might decrease but correctness increases. Doesn't this break the collapse theorem?*

Response 2. *This objection confirms rather than refutes our thesis. Consider what "selecting the best solution" requires:*

1. *A* verifier *or* reward function *that evaluates solution quality.*
2. *This verifier must encode knowledge of what constitutes a "good" solution.*
3. *This knowledge was designed by humans.*

In AlphaZero [8], the verifier is the game rules—a compact, perfect, human-designed evaluation function. In RLHF [12], the reward model is trained on human preferences. In best-of-N sampling with a code interpreter, the interpreter was human-engineered.

Every filtering mechanism that breaks distributional collapse does so by introducing an external signal. By Definition 1.5, this constitutes human influence. Our proof explicitly addresses

the case where no such external signal exists. If you need filtering to avoid collapse, you need human influence to avoid collapse. That is our point.

Formally, let $\mathcal{R} : \mathcal{V}^* \to \mathbb{R}$ be a reward function and define the filtered distribution:

$$p^{\mathcal{R}}_{\boldsymbol{\theta}_k}(\boldsymbol{x}) \propto p_{\boldsymbol{\theta}_k}(\boldsymbol{x}) \cdot \exp(\beta \cdot \mathcal{R}(\boldsymbol{x})) \tag{18}$$

The information in $p^{\mathcal{R}}_{\boldsymbol{\theta}_k}$ is bounded by:

$$I(p^{\mathcal{R}}_{\boldsymbol{\theta}_k}; p_{\text{true}}) \leq I(p_{\boldsymbol{\theta}_k}; p_{\text{true}}) + I(\mathcal{R}; p_{\text{true}}) \tag{19}$$

The term $I(\mathcal{R}; p_{\text{true}})$ is strictly positive—the reward function contributes new information about ground truth. But this information originates from human design. Remove the human-designed $\mathcal{R}$ and you are back to unfiltered self-distillation, which collapses by Theorem 3.1.

4 Gödelian Limits on Self-Verification

Theorem 4.1 (Impossibility of Autonomous Self-Verification)**.** *Let $\mathcal{M}_k$ denote the Turing machine equivalent of $\boldsymbol{\theta}_k$. The model cannot verify that its self-modifications constitute genuine improvements.*

Proof. Define IMPROVE$(k) \iff \mathcal{Q}(\boldsymbol{\theta}_{k+1}) > \mathcal{Q}(\boldsymbol{\theta}_k)$ for any quality metric $\mathcal{Q}$ correlating with progress toward AGI.

Gödel's First Incompleteness Theorem: $\mathcal{M}_k$, being a formal system capable of arithmetic, contains true statements it cannot prove:

$$\exists\, \phi_k : \quad \mathcal{M}_k \nvdash \phi_k \quad \text{and} \quad \mathcal{M}_k \nvdash \neg\phi_k \tag{20}$$

Statements about whether $\boldsymbol{\theta}_{k+1}$ will fail on inputs outside $\text{supp}(\mathcal{D}^{\text{self}}_k)$ may be among the undecidable statements.

Gödel's Second Incompleteness Theorem: The model cannot prove its own consistency:

$$\mathcal{M}_k \nvdash \text{Con}(\mathcal{M}_k) \tag{21}$$

Rice's Theorem: Any non-trivial semantic property of the function computed by $\mathcal{M}_{k+1}$ is undecidable by any Turing machine.

We emphasize that these results do not prevent *all* verification—a model can check specific test cases, run benchmarks, and verify particular outputs. What they prevent is *complete* verification: the model cannot guarantee that $\boldsymbol{\theta}_{k+1}$ is globally better than $\boldsymbol{\theta}_k$ across all possible inputs. Since AGI requires correctness across all computable functions (Definition 1.7), partial verification is insufficient. The model can confirm improvement on tested cases while unknowingly degrading on untested ones—and it cannot determine, from within itself, whether this is happening. □

Corollary 4.2 (The Verification Regress)**.** *If $\mathcal{M}_k$ builds a verifier $\mathcal{V}_k$, then $\mathcal{V}_k$ is itself a formal system subject to the same incompleteness. Verifying $\mathcal{V}_k$ requires $\mathcal{V}'_k$, generating an infinite regress:*

$$\mathcal{M}_k \to \mathcal{V}_k \to \mathcal{V}'_k \to \mathcal{V}''_k \to \cdots \tag{22}$$

Each link in the chain inherits the incompleteness of its predecessor and adds its own statistical error.

4.1 Why Gödel Does Not Apply Symmetrically to Humans

Objection 3 (The Human Symmetry Argument)**.** *If Gödelian limits prevent machines from self-improving, they should equally prevent humans, since humans are also finite systems. But humans clearly self-improve (from pre-linguistic hominids to mathematicians). Therefore the Gödelian argument proves too much.*

Response 3. *This objection fails because it assumes humans are formal systems. We argue they are not, for the following reasons.*

The LLM is provably a formal system. *An LLM consists of:*

- *Finite-precision floating-point arithmetic (float16, bfloat16, or float32),*
- *Deterministic matrix multiplications, additions, and nonlinear activations,*
- *Pseudo-random number generation for sampling (algorithmically deterministic given a seed).*

Every operation is computable. The entire system is realizable as a Turing machine. Gödel's theorems apply without qualification.

The human brain is not a formal system. *The brain is a quantum-biological system:*

- ***Synaptic transmission*** *involves probabilistic vesicle release governed by quantum tunneling of Ca^{2+} ions through voltage-gated channels.*
- ***Ion channel dynamics*** *are inherently quantum mechanical—channel state transitions are stochastic processes with irreducible quantum uncertainty.*
- ***Molecular processes*** *underlying neural computation (protein folding, neurotransmitter binding, enzymatic activity) operate in the quantum regime.*

If any of these quantum processes contribute to cognitive state transitions, then the brain has access to genuine ontological randomness—not pseudo-randomness generated by an algorithm, but irreducible indeterminacy as characterized by quantum mechanics.

Formal consequence: *Let $\hat{\rho}_t$ be the quantum density operator describing the brain's state. If the decision process depends on quantum measurement outcomes, then the brain's output is not computable by any Turing machine (since quantum measurement outcomes are fundamentally non-algorithmic). A system that is not a Turing machine is not a formal system in Gödel's sense, and the incompleteness theorems do not apply.*

The asymmetry:

$$\underbrace{\textit{LLM}}_{\textit{formal system}} \xrightarrow{\textit{Gödel applies}} \textit{incomplete, cannot fully self-verify} \tag{23}$$

$$\underbrace{\textit{Human brain}}_{\textit{quantum-biological}} \xrightarrow{\textit{Gödel does not apply}} \textit{not subject to these bounds} \tag{24}$$

This does not require accepting Penrose's specific microtubule hypothesis [10]. It requires only the weaker claim that quantum effects at the molecular level influence neural computation—a claim supported by the physics of ion channels, synaptic vesicle release, and molecular binding, all of which operate at scales where quantum effects are non-negligible.

Therefore, the fact that humans can self-improve does not imply that formal computational systems can. The situations are not symmetric.

5 The SGD Fixed-Point Trap

Theorem 5.1 (Self-Training Converges to a Trivial Fixed Point)**.** *Under self-training, SGD converges to a fixed point where the model reproduces its own output distribution.*

Proof. The self-training loss is:

$$\mathcal{L}_k(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{x}\sim p_{\boldsymbol{\theta}_k}}\left[\frac{1}{T}\sum_{t=1}^{T}\log p_{\boldsymbol{\theta}}(x_t \mid x_{<t})\right] = H(p_{\boldsymbol{\theta}_k}) + \mathrm{KL}(p_{\boldsymbol{\theta}_k}\|p_{\boldsymbol{\theta}}) \tag{25}$$

This is minimized when $p_{\boldsymbol{\theta}} = p_{\boldsymbol{\theta}_k}$ (where KL = 0). Therefore $\boldsymbol{\theta}_k$ is already a global minimum of $\mathcal{L}_k$. SGD converges back to (approximately) $\boldsymbol{\theta}_k$.

The Fisher information matrix captures the effective dimensionality of optimization:

$$\boldsymbol{F}_k = \mathbb{E}_{\boldsymbol{x}\sim p_{\boldsymbol{\theta}_k}}\left[\nabla_{\boldsymbol{\theta}}\log p_{\boldsymbol{\theta}}(\boldsymbol{x})\,\nabla_{\boldsymbol{\theta}}\log p_{\boldsymbol{\theta}}(\boldsymbol{x})^\top\right]\Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k} \tag{26}$$

As $p_{\boldsymbol{\theta}_k}$ concentrates (Theorem 3.1), $\mathrm{rank}(\boldsymbol{F}_k)$ decreases. The model's gradients become confined to a shrinking subspace. Optimization can only rearrange existing capabilities within this subspace, not generate novel ones. □

Remark 5.2. *This formalizes the intuition: a model training on its own output is trying to become better at being itself. The loss function rewards reproducing what it already does. This is not self-improvement; it is self-reinforcement.*

6 The Pattern Completion Barrier

Theorem 6.1 (Separation Between Pattern Completion and General Reasoning)**.** *There exist computable functions $f \in \mathcal{F}_{\mathrm{AGI}}$ that are not approximable by statistical pattern completion on any finite training set.*

Proof. We exhibit three classes of functions that are computable but unreachable by pattern matching:

(a) Gödel-undecidable theorems: For any consistent formal system $\mathcal{F}$ (including the implicit system represented by all mathematical text in $\mathcal{D}_{\mathrm{train}}$), there exist true mathematical statements not provable within $\mathcal{F}$. Their proofs cannot be found by pattern-matching on existing proofs.

(b) Kolmogorov-complex algorithms [13]: Let f be a function with Kolmogorov complexity $K(f) > K_{\max}(\mathcal{D}_{\mathrm{train}})$. Then f cannot be reconstructed from patterns in $\mathcal{D}_{\mathrm{train}}$.

(c) Empirical facts: Knowledge about the physical world not recorded in $\mathcal{D}_{\mathrm{train}}$ and not logically derivable from it requires observation, which a closed-loop text system cannot perform. □

Objection 4 (Compositionality)**.** *Even if a program has complexity greater than any single training example, it may be derivable by combining patterns. Humans learn math from examples but prove new theorems.*

Response 4. *Compositionality allows* recombination *of existing patterns, which is a form of search over the space of combinations. This is addressed by the information ceiling (Theorem 2.1): the total information available for composition is bounded by the training data. Composition can extract latent information more efficiently, but it cannot exceed the information-theoretic ceiling.*

Furthermore, the human analogy fails for the same reason as the Gödel symmetry argument (Section 4.1): humans are quantum-biological systems not bound by the formal limitations of Turing machines. A human mathematician proving a new theorem may be accessing cognitive processes that are not pattern completion at all, but involve genuine creativity grounded in non-computable physical processes.

A transformer is theoretically *Turing-complete only with unbounded computation time and unbounded memory. In practice, it has a fixed context window C (typically $\leq 2^{17}$ tokens) and fixed precision, making it a finite automaton, strictly weaker than a Turing machine.*

7 The Computational Complexity Barrier (P $\neq$ NP)

Theorem 7.1 (Self-Improvement Requires Solving NP-Hard Problems)**.** *Each step of autonomous self-improvement requires the model to solve an optimization problem over its parameter space that is NP-hard in general. If* P $\neq$ NP*, no polynomial-time algorithm (including the model itself) can reliably find optimal or near-optimal solutions.*

Proof. At each self-improvement step k, the model must find:

$$\boldsymbol{\theta}^*_{k+1} = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^p} \mathcal{L}_{\text{true}}(\boldsymbol{\theta}) \tag{27}$$

where $\mathcal{L}_{\text{true}}$ is the loss relative to ground truth. This is a non-convex optimization problem over $\mathbb{R}^p$ with $p \sim 10^9$ to 10^{12} dimensions. Non-convex optimization over continuous spaces with discrete structural constraints is NP-hard in general, as is the closely related problem of training neural networks to global optimality.

The key structural feature is the asymmetry between verification and search:

- **Verification** (checking if a given $\boldsymbol{\theta}_{k+1}$ improves on $\boldsymbol{\theta}_k$): compute $\mathcal{L}(\boldsymbol{\theta}_{k+1})$ and compare. This requires $O(p \cdot C \cdot L)$ operations—polynomial in model size.
- **Search** (finding a $\boldsymbol{\theta}_{k+1}$ that improves on $\boldsymbol{\theta}_k$): explore a space of size $O(2^{p \cdot b})$ (all possible parameter configurations at b-bit precision). In the worst case, this is exponential.

This is precisely the P vs. NP structure: verification is polynomial, but search is (under P $\neq$ NP) superpolynomial.

Stochastic gradient descent provides a heuristic that avoids exhaustive search by following local gradients. However, SGD:

1. Converges to local minima, not global minima,
2. Cannot escape basins of attraction without external perturbation,
3. On self-generated data, converges to the trivial fixed point $\boldsymbol{\theta}_k$ (Theorem 5.1).

Finding the global optimum—or even a substantially better local optimum—requires solving the NP-hard search problem that SGD explicitly avoids. □

Theorem 7.2 (LLMs as Empirical Evidence for P $\neq$ NP)**.** *The failure of LLMs to solve NP-hard problems despite massive statistical exposure constitutes strong empirical evidence that* P $\neq$ NP*, which in turn strengthens the self-improvement impossibility result.*

Proof. An LLM trained on the entirety of available human text has been exposed to billions of instances of solved combinatorial problems—traveling salesman solutions, scheduling solutions, graph coloring solutions, satisfiability instances, cryptographic challenges. If polynomial-time statistical shortcuts existed for NP-hard problems—that is, if there were learnable patterns in the structure of NP solutions—then a sufficiently large pattern-matching system trained on sufficiently many examples would discover them.

LLMs are the largest pattern-matching systems ever constructed, trained on the largest corpora ever assembled. They have had more statistical exposure to solved NP instances than any system in history. Despite this:

1. LLMs fail to solve large instances of the traveling salesman problem.
2. LLMs cannot factor large semiprimes.
3. LLMs cannot find optimal schedules for non-trivial constraint satisfaction problems.
4. LLMs cannot reliably solve novel SAT instances beyond small sizes.

The model can *recognize* what a correct solution looks like (verification) but cannot *find* one efficiently (search). This gap between recognition and generation is precisely the P $\neq$ NP gap made empirically observable.

If P = NP, the statistical regularities enabling polynomial-time solutions would be present in the data and learnable by a sufficiently powerful pattern matcher. The LLM's failure to learn them, despite unprecedented scale and data, is evidence that these regularities do not exist. □

Corollary 7.3 (P $\neq$ NP Strengthens the Information Ceiling)**.** *Even information that is technically "latent" in the training data (the information ceiling of Theorem 2.1) may be computationally inaccessible. Under* P $\neq$ NP*, extracting certain latent information requires superpolynomial computation:*

$$\exists\, f \in \mathrm{NP}: \quad f \text{ is derivable from } \mathcal{D}_{\text{train}} \text{ but requires } 2^{\Omega(n)} \text{ steps to extract} \tag{28}$$

Therefore, the effective information available to the model is strictly less than the theoretical information ceiling:

$$I_{\text{accessible}}(\boldsymbol{\theta}_k; p_{\text{true}}) < I(\boldsymbol{\theta}_k; p_{\text{true}}) \leq I(\mathcal{D}_{\text{train}}; p_{\text{true}}) \tag{29}$$

The information ceiling was already a hard limit. Computational complexity makes the effective *ceiling even lower.*

Remark 7.4 (The Self-Improvement Loop as Iterated NP-Hard Search)**.** *Each iteration of autonomous self-improvement requires solving an NP-hard optimization problem (finding better parameters). The model's tool for this search is statistical pattern matching. Its own failure to crack NP-hard problems—despite being the most powerful pattern matcher ever built—is evidence that this tool is fundamentally inadequate for the search required. The model is being asked to do at every self-improvement step the very thing it demonstrably cannot do. This is not a limitation that scales away: if* P $\neq$ NP*, no amount of additional parameters, data, or compute converts the polynomial verification capability of an LLM into the superpolynomial search capability that self-improvement demands.*

8 The Context-Computation Paradox

Theorem 8.1 (Self-Correction Requires Unbounded Resources)**.** *Let $\boldsymbol{\theta}_k \in \mathbb{R}^p$ be the model parameters at step k, with p on the order of 10^9 to 10^{12}. Let C be the context window size (maximum number of tokens processable in a single forward pass). For the model to verify or correct its own parameters, it must represent and reason about the state space $\mathbb{R}^p$. The following bound holds:*

$$\frac{p \cdot b}{C \cdot \log_2 |\mathcal{V}|} \gg 1 \tag{30}$$

where b is the number of bits per parameter (typically 16 or 32). That is, the information required to describe the model's own parameter space vastly exceeds the model's processing capacity in any single reasoning step.

Proof. A model with $p = 10^{11}$ parameters at 16-bit precision contains 1.6×10^{12} bits of state information. With a context window of $C = 128{,}000$ tokens and vocabulary size $|\mathcal{V}| \approx 10^5$, each context window carries approximately $128{,}000 \times 17 \approx 2.2 \times 10^6$ bits of information.

The ratio:

$$\frac{\text{model state}}{\text{context capacity}} \approx \frac{1.6 \times 10^{12}}{2.2 \times 10^6} \approx 7.3 \times 10^5 \tag{31}$$

The model would need approximately 730,000 sequential reasoning steps to even *read* its own parameters, let alone reason about them.

For self-correction, the model must:

1. Identify which parameters contribute to errors (requires gradient computation across the full parameter space),
2. Determine correct parameter values (requires optimization over $\mathbb{R}^p$),
3. Verify that corrections do not introduce new errors (requires evaluation across all possible inputs).

Each step requires processing information proportional to p or larger. Since the model can process at most C tokens per step, the number of required steps grows as $\Omega(p/C)$ at minimum. For verification across all inputs (step 3), the required computation is unbounded.

Therefore, even if the model could in principle self-correct (which Theorem 4.1 shows it cannot do completely), the computational resources required grow without bound relative to its finite processing capacity:

$$\lim_{p\to\infty} \frac{\text{Resources required for self-correction}}{\text{Resources available per step}} = \infty \tag{32}$$

□

Theorem 8.2 (The Overhead Paradox)**.** *Even in the theoretical best case where the context window C is extended to equal the full parameter representation ($C \cdot \log_2 |\mathcal{V}| = p \cdot b$), self-correction remains impossible due to computational overhead that creates an irreducible deficit.*

Proof. Suppose we grant the model a context window large enough to hold its entire parameter state. The context must then be partitioned among competing demands:

$$C = C_{\text{state}} + C_{\text{reasoning}} + C_{\text{temp}} + C_{\text{output}} \tag{33}$$

where:

- C_{state}: tokens representing the parameters to be examined,
- $C_{\text{reasoning}}$: tokens for the logical operations analyzing those parameters (comparisons, gradient estimation, error diagnosis),
- C_{temp}: tokens for intermediate results and working storage,
- C_{output}: tokens for the corrected parameter specifications.

Since $C_{\text{state}} = C$ (by assumption, the model's full state fills the context), we require:

$$C_{\text{reasoning}} + C_{\text{temp}} + C_{\text{output}} > 0 \tag{34}$$

for any non-trivial reasoning to occur. But $C_{\text{state}} + C_{\text{reasoning}} + C_{\text{temp}} + C_{\text{output}} > C$, which is a contradiction.

In practice, meaningful self-analysis requires substantial reasoning overhead. At minimum, $C_{\text{reasoning}} \geq \alpha \cdot C_{\text{state}}$ for some constant $\alpha > 0$ reflecting the complexity of error diagnosis. Therefore:

$$C_{\text{required}} \geq (1+\alpha) \cdot C_{\text{state}} + C_{\text{temp}} + C_{\text{output}} > C \tag{35}$$

The deficit is irreducible: *any* system reasoning about a representation of itself requires more resources than the representation alone occupies. □

Theorem 8.3 (The Scaling Trap)**.** *Increasing the context window C to accommodate self-reasoning does not resolve the deficit, because model complexity grows with context capacity.*

Proof. Let C_k be the context window at self-improvement step k, and p_k the parameter count. In practice, model capability scales with both p and C, and the relationship is approximately:

$$p_k = g(C_k), \quad \text{where } g \text{ is monotonically increasing} \tag{36}$$

(larger context windows require more parameters to effectively utilize the additional context—embedding layers scale as $O(C \cdot d)$, attention heads must increase, etc.). Therefore the ratio:

$$R_k = \frac{p_k \cdot b}{C_k \cdot \log_2 |\mathcal{V}|} \tag{37}$$

does not converge to 1 as C_k increases. Empirically, R_k remains large or grows, because $g(C_k)$ grows at least linearly in C_k.

Even setting aside this architectural coupling, the *computational cost* of processing context C in the standard transformer is:

$$\text{FLOPS}_{\text{attention}} = O(C^2 \cdot d) \tag{38}$$

due to the quadratic scaling of self-attention. Doubling C quadruples the compute required per forward pass. The time to perform one self-reasoning step therefore grows as:

$$T_{\text{step}}(C) = O(C^2) \tag{39}$$

For a model attempting to reason about its full parameter space ($C \propto p$), the time per self-correction step is:

$$T_{\text{step}} = O(p^2) \tag{40}$$

For $p = 10^{11}$, this yields $T_{\text{step}} \propto 10^{22}$ FLOPS per reasoning step. At current hardware speeds ($\sim 10^{18}$ FLOPS/s for the largest clusters), a single self-reasoning pass would take approximately 10^4 seconds ≈ 3 hours—for one step of what would require millions of sequential steps to cover the parameter space. □

Theorem 8.4 (Training Time Divergence)**.** *The time required for the model to retrain itself after identifying corrections grows superlinearly with model size, creating a secondary resource explosion.*

Proof. Suppose the model, at self-improvement step k, has identified a set of corrections to apply to its parameters. Implementing these corrections requires a training process (gradient-based optimization). A single training step requires:

$$T_{\text{train_step}} = O(p \cdot C \cdot L) \tag{41}$$

FLOPS, where L is the number of transformer layers. Convergence to a new parameter configuration typically requires S training steps, where S depends on the magnitude of the correction and the loss landscape curvature. The total training time is:

$$T_{\text{train_total}} = S \cdot O(p \cdot C \cdot L) \tag{42}$$

For the self-improvement loop, this training must occur at *every* iteration k. The total compute for K iterations of self-improvement is:

$$T_{\text{total}} = \sum_{k=0}^{K-1} \left[\underbrace{T_{\text{diagnose}}(k)}_{\text{identify errors}} + \underbrace{T_{\text{train}}(k)}_{\text{apply corrections}} + \underbrace{T_{\text{verify}}(k)}_{\text{check results}} \right] \tag{43}$$

By Theorem 5.1, the achievable improvement $\Delta_k \to 0$, meaning each successive iteration produces diminishing returns. But the *cost* per iteration does not diminish—it remains $\Omega(p \cdot C \cdot L)$ per training cycle. The ratio of cost to benefit therefore diverges:

$$\frac{T_{\text{train}}(k)}{\Delta_k} \to \infty \quad \text{as } k \to \infty \tag{44}$$

Furthermore, as errors accumulate (Theorem 9.1), the corrections required become more complex (affecting larger portions of the parameter space), increasing S_k. Simultaneously, the model's ability to correctly diagnose errors degrades (since it is using a progressively more corrupted model to identify corruption). This creates a feedback loop:

1. More errors $\implies$ more parameters need correction $\implies$ longer training,
2. Longer training on self-generated data $\implies$ more distributional collapse (Theorem 3.1),
3. More collapse $\implies$ worse error diagnosis $\implies$ more errors introduced.

The self-improvement process is therefore not merely slow but *anti-convergent*: the resources required per unit of improvement grow without bound, while the quality of each improvement attempt simultaneously degrades. In the limit:

$$\lim_{k\to\infty} \frac{\text{Compute cost of iteration } k}{\text{Genuine improvement from iteration } k} = \infty \tag{45}$$

Even with unbounded time and compute, the process does not converge—it diverges, because the errors introduced at each step eventually exceed the corrections achieved. □

Remark 8.5 (The Infinite Loop). *Theorems 8.1–8.4 together describe an inescapable recursive trap:*

1. *The model cannot fit a representation of itself into its working memory (8.1).*
2. *Even if it could, the overhead of reasoning about that representation exceeds the available space (8.2).*
3. *Expanding the space increases the model's complexity proportionally, preserving the deficit (8.3).*
4. *The time to retrain after each correction cycle diverges relative to the improvement gained (8.4).*

Every attempt to provide the system with sufficient resources for self-correction generates a new, larger resource requirement. The gap between what the system needs and what it has is not a fixed deficit to be closed by engineering—it is a structural property that scales with any attempt to close it. The process requires infinite time and infinite resources, and even then does not converge due to the error compounding of Theorem 9.1. It is, in the precise mathematical sense, an infinite loop with no halting condition.

9 Error Compounding in Recursive Self-Modification

Theorem 9.1 (Error Divergence Without External Correction). *Let ϵ_k measure the error of $\boldsymbol{\theta}_k$ relative to ground truth. At each step:*

$$\epsilon_{k+1} = \epsilon_k - \Delta_k + \delta_k + \sum_{j=0}^{k} \alpha_{k,j}\, \delta_j \tag{46}$$

where $\Delta_k \geq 0$ is the intended improvement, $\delta_k > 0$ is the undetectable degradation (which is strictly positive by Theorem 4.1—the model cannot fully verify its modifications), and $\alpha_{k,j} \geq 0$ captures error propagation through self-generated training data.

Since errors propagate through the data chain (each $\mathcal{D}_k^{\text{self}}$ inherits all errors in $\boldsymbol{\theta}_k$), and since the improvement $\Delta_k \to 0$ as the model approaches its fixed point (Theorem 5.1), while δ_k remains bounded away from zero:

$$\liminf_{k\to\infty} \epsilon_k = \infty \tag{47}$$

The error diverges. Without an external correction mechanism (human verification), the model degrades.

Remark 9.2 (The Statistical Error Floor). *The fundamental issue is that the system is stochastic. Every output has an associated error probability. When the system uses its own outputs to train itself, it trains on data that contains errors. The next generation inherits those errors and adds its own. The error is cumulative and the correction mechanism (the model itself) has the same statistical error rate as the system it is trying to correct. This is not a bug to be fixed; it is a mathematical property of using statistics to correct statistics.*

An external corrector (human or human-designed verifier) breaks this cycle because its error properties are independent of the model's. But an AI-generated corrector has correlated errors—it was built by the same flawed process—and therefore cannot provide the independent error correction needed.

10 Main Theorem

Theorem 10.1 (Impossibility of Autonomous LLM Self-Improvement to AGI)**.** *Let $\{p_{\boldsymbol{\theta}_k}\}_{k=0}^{\infty}$ be the sequence of models produced by autonomous self-improvement (Definition 1.6). Then $\{p_{\boldsymbol{\theta}_k}\}$ cannot converge to AGI (Definition 1.7).*

1. ***Information ceiling*** *(Theorem 2.1): Self-generated data cannot create new information about ground truth. Inference-time search can better utilize existing information but is bounded by the same ceiling. The model's knowledge is capped at what the training data contained.*

2. ***Distributional collapse*** *(Theorem 3.1): Without external filtering (which constitutes human influence), self-training monotonically contracts entropy and support. The model becomes less general. Every mechanism that prevents collapse (reward functions, selection criteria, external verifiers) injects human-designed intelligence into the loop.*

3. ***Gödelian verification failure*** *(Theorem 4.1): The model cannot completely verify its self-modifications. Partial verification (benchmarks, test cases) is possible but insufficient for AGI, which requires correctness across all computable functions. The Gödelian limitation applies asymmetrically: LLMs are provably formal systems subject to incompleteness, while human brains are quantum-biological systems that are not formal systems in Gödel's sense.*

4. ***SGD fixed-point trap*** *(Theorem 5.1): Self-training converges to reproducing the model's own distribution. The Fisher information rank contracts, confining optimization to a shrinking subspace.*

5. ***Pattern completion barrier*** *(Theorem 6.1): Stochastic pattern completion on finite training data is provably weaker than general computation. Compositionality enables recombination but remains bounded by the information ceiling. Practical transformers are finite automata, not Turing machines.*

6. ***Computational complexity barrier*** *(Theorems 7.1–7.2, Corollary 7.3): Each self-improvement step requires solving an NP-hard optimization problem over the parameter space. The model's tool for this search is statistical pattern matching—the same tool that demonstrably fails on NP-hard problems despite unprecedented scale and data exposure. If* $\mathrm{P} \neq \mathrm{NP}$*, this gap between verification (polynomial) and search (superpolynomial) is permanent. Furthermore, computational complexity makes the effective information ceiling strictly lower than the theoretical one: latent information may exist in the weights but be computationally inaccessible.*

7. ***Context-computation paradox*** *(Theorems 8.1–8.4): The model's working memory is orders of magnitude too small to represent its own parameter space. Even with unlimited context, computational overhead for reasoning about the representation exceeds available space (Theorem 8.2). Expanding resources increases model complexity proportionally, preserving the deficit (Theorem 8.3). Training time per correction cycle diverges relative to improvement gained, and the process becomes anti-convergent: more errors require longer correction, longer correction causes more collapse, more collapse causes worse error diagnosis (Theorem 8.4). The process forms an infinite loop with no halting condition.*

8. ***Error divergence*** *(Theorem 9.1): Statistical errors compound through recursive self-training. The correction mechanism (the model) has the same error rate as the system being corrected. An AI-built corrector inherits correlated errors from the same flawed process. Only an external corrector with independent error properties can break the cycle.*

Therefore:

$$\boxed{\forall\, \boldsymbol{\theta}_0 \in \mathbb{R}^p,\ \ \forall\, k \geq 0: \quad \mathcal{S}^k(\boldsymbol{\theta}_0) \not\to \boldsymbol{\theta}^*_{\mathrm{AGI}}} \tag{48}$$

within the autonomous stochastic autoregressive framework. □

11 Discussion

11.1 What the Proof Shows

This proof establishes that a stochastic language model, operating without any form of human influence, cannot self-improve to AGI. The result holds for any model size, any architecture within the autoregressive transformer family, and any training schedule. Scale provides quantitative improvements within a bounded function class but does not alter the qualitative impossibility.

11.2 What Would Escape These Bounds

A system escapes these bounds if it incorporates:

- External information sources (new data, environment interaction) — violates closed-loop assumption,
- Human-designed evaluation signals (reward functions, preference models) — constitutes human influence,
- Human-engineered tools or verifiers (code interpreters, proof checkers, game rules) — constitutes human influence,
- Non-stochastic components (formal theorem provers, symbolic reasoning engines) — departs from the stochastic framework.

Each of these either introduces human influence or leaves the stochastic paradigm. In either case, the system is no longer what the singularity hypothesis describes: an AI that autonomously bootstraps itself to superintelligence without human involvement.

11.3 The Stochastic Parrot, Formalized

Our results provide mathematical grounding for the characterization of LLMs as "stochastic parrots" [6]. The model is a conditional probability estimator: $p_{\boldsymbol{\theta}}(x_t \mid x_{<t})$. Self-improvement within this framework is self-distillation: the parrot learning to parrot itself. The output may become more fluent, more consistent, more confident—but it cannot become more *correct* about things absent from training, more *creative* beyond recombination of observed patterns, or more *general* than the information-theoretic ceiling permits.

11.4 Human Influence Is Provably Necessary

The central conclusion is not that LLMs cannot improve—they clearly can and do. It is that *human influence is a mathematically necessary component of that improvement.* Whether through curated training data, designed reward functions, engineered tools, or architectural innovations, the intelligence driving LLM progress originates in human cognition. The singularity scenario [11]—in which this dependency is severed and the AI improves autonomously—requires violations of the data processing inequality, Gödel's incompleteness theorems, and fundamental properties of stochastic optimization. It is not an engineering challenge. It is a mathematical impossibility within the framework.

11.5 A Control Theory Perspective

Control theory provides an intuitive framing for the central impossibility. A feedback control system works because the controller is *independent* of the plant. A thermostat is not made of

the same heat it regulates.

An LLM attempting to regulate itself is simultaneously the plant and the controller. The sensor, actuator, and error estimator are all the same stochastic process with the same systematic biases. When the system drifts, the component detecting the drift has drifted by the same amount in the same direction.

In control theory terms, the controller's noise is coupled to the plant's noise with correlation approaching 1. This violates the fundamental assumption that makes feedback control work: independent error correction.

More precisely, classical control theory requires four prerequisites that LLMs violate:

1. **Observable state.** The meaningful state of an LLM is the distribution $p_{\boldsymbol{\theta}}$ over all possible inputs—a function over an exponentially large space. This state is not observable; it can only be sampled. No finite number of samples characterizes the distribution.
2. **Plant model.** The relationship between parameter changes $\Delta\boldsymbol{\theta}$ and behavioral changes $\Delta p_{\boldsymbol{\theta}}$ is nonlinear, non-convex, and not characterizable in closed form. No transfer function exists. Without a plant model, no controller can compute corrective inputs.
3. **Measurable error signal.** Feedback control requires $e(t) = r(t) - y(t)$, where $r(t)$ is a reference signal. The reference signal for "general intelligence" would require a complete specification of AGI—which loops back to the verification problem of Theorem 4.1.
4. **Stability guarantees.** Lyapunov stability and BIBO stability require structural properties (linearity, bounded gain, known dynamics) that LLM training dynamics lack. The self-training loop has no Lyapunov function guaranteeing convergence.

A PID controller on a motor works because you can measure the angle, you know the transfer function, and the error signal is trivially defined. An LLM fails on all three. The system is unobservable, uncharacterizable, and has no well-defined reference signal. Self-regulation under these conditions is not an engineering problem to be solved—it is a category error.

11.6 The Multi-Agent Correlation Problem

Objection 5 (Multiple Agents Checking Each Other)**.** *What if we deploy multiple LLM agents in a pipeline—an "architect" that diagnoses problems and a "programmer" that implements fixes, or a "generator" and a "critic"—so that they provide independent error correction for each other?*

Response 5. *This does not work because the agents' errors are correlated. Multiple agents derived from the same (or similar) training distributions share:*

- *The same training data $\implies$ the same blind spots,*
- *The same pretraining objective $\implies$ the same systematic biases,*
- *The same failure modes on edge cases and distribution-boundary inputs.*

Let δ_{gen} be the error introduced by the generator and δ_{crit} be the error in the critic's evaluation. For independent correction, we need:

$$\text{Corr}(\delta_{\text{gen}}, \delta_{\text{crit}}) \approx 0 \tag{49}$$

But since both agents are functions of the same training distribution $\mathcal{D}_{\text{train}}$:

$$\text{Corr}(\delta_{\text{gen}}, \delta_{\text{crit}}) \to 1 \quad \textit{as overlap in training data} \to 1 \tag{50}$$

When the generator introduces a subtle bug, the critic has the same blind spot because it learned from the same distribution. The errors resonate*—they reinforce rather than cancel.*

This generalizes to any number of agents: n agents with correlated errors provide strictly less than n independent checks. In the limit of identical training distributions, n agents provide exactly one independent perspective, regardless of n. The correlation is not in the architecture—it is in the training distribution. You cannot escape it by factoring the agent into components.

The SWE-CI benchmark [7] provides direct empirical evidence: its dual-agent architecture (Architect + Programmer) still exhibits regressions in over 75% of tasks despite the agents checking each other (see Section 11.8).

The only way to decorrelate errors is to introduce a signal from outside the distribution—which is human influence by Definition 1.5.

11.7 Why Noise Cannot Substitute for Human Influence

A natural objection to the multi-agent correlation problem is: "Add randomness to decorrelate the agents." This fails because of Theorem 2.1.

White noise is statistically independent of the model's errors but carries zero mutual information with ground truth:

$$I(\text{noise}; p_{\text{true}}) = 0 \tag{51}$$

Adding noise to the model's output:

$$I(p_{\boldsymbol{\theta}_k} + \text{noise}; p_{\text{true}}) \leq I(p_{\boldsymbol{\theta}_k}; p_{\text{true}}) \tag{52}$$

by the data processing inequality. Information is *lost*, not gained. The noise decorrelates the agents' errors in random directions, not toward correctness. The result is agents that disagree more often, but not agents that disagree *usefully*.

What is needed is not independence alone—it is *informed* independence. The external corrector must have mutual information with ground truth that is statistically independent of the model's errors. A human reviewer satisfies this: their knowledge originates from a completely different process (biological cognition, real-world embodied experience) and correlates with ground truth through a different channel.

In control theory terms (Section 11.5): adding noise to the sensor does not improve the state estimate. What is needed is a *second sensor* that measures the actual system through a different physical principle. Human oversight is that second sensor.

11.8 Empirical Corroboration: The SWE-CI Benchmark

Independent empirical evidence for the theoretical results of this paper is provided by SWE-CI (Software Engineering—Continuous Integration) [7], the first repository-level benchmark evaluating LLM agent capabilities in maintaining codebases over iterative CI loops.

11.8.1 Experimental Setup

SWE-CI comprises 100 tasks drawn from 68 real-world repositories, with each task spanning an average of 233 days and 71 consecutive commits of authentic evolutionary history. The benchmark evaluates 18 models from 8 providers, consuming over 10 billion tokens. The iteration cycle—run tests, define requirements, modify code, repeat—instantiates precisely the autonomous self-improvement loop of Definition 1.6, applied to code rather than model weights.

SWE-CI employs an Architect–Programmer dual-agent protocol: the Architect identifies functional gaps and issues requirements; the Programmer implements them. This is an explicit attempt at independent error correction through role decomposition.

11.8.2 Key Result: Zero-Regression Rate

The zero-regression rate measures the fraction of tasks in which the agent never regresses—that is, never breaks a previously passing test throughout the entire maintenance process. This metric directly corresponds to whether $\delta_k > 0$ (undetectable degradation) occurs in our error model (Theorem 9.1).

Result: Most models achieve a zero-regression rate below 0.25. In over 75% of tasks, the agent introduces at least one regression. Only two models in the Claude Opus series exceed a zero-regression rate of 0.5, meaning even the best models regress in roughly half of all tasks. This is a direct, large-scale measurement of $\delta_k > 0$.

11.8.3 Mapping to Theoretical Results

Theorem	SWE-CI Evidence
9.1 (Error Divergence)	Zero-regression rate < 0.25 for most models: regressions occur in 75%+ of tasks, directly measuring $\delta_k > 0$
3.1 (Distributional Collapse)	Agent quality degrades over longer iteration sequences; EvoScore with $\gamma > 1$ penalizes models that accumulate technical debt
Sec. 11.6 (Correlated Corrector)	Dual-agent architecture (Architect + Programmer) still fails at comparable rates—role decomposition does not provide independent error correction
Sec. 11.5 (Human Influence)	Model improvement across the evaluation period comes only from human-driven releases (new architectures, training data, RLHF), not from autonomous iteration

Table 1: Mapping between theoretical impossibility results and SWE-CI empirical observations.

11.8.4 Within-Run Degradation vs. Across-Release Improvement

SWE-CI's results reveal a striking asymmetry. *Within* an autonomous run (the CI iteration loop), quality degrades: regressions accumulate, technical debt compounds, and the agent's ability to maintain the codebase deteriorates over successive iterations. *Across* model releases (Claude Sonnet to Claude Opus, GPT-4o to GPT-o3, etc.), performance improves steadily.

This asymmetry is precisely what the theory predicts. Autonomous iteration (no human influence) leads to error divergence (Theorem 9.1) and distributional collapse (Theorem 3.1). Inter-release improvement reflects human-driven development: new architectures, curated training data, human-designed reward functions, and engineering effort. The intelligence driving progress across releases originates in human cognition—confirming that human influence is provably necessary (Section 10.4).

The SWE-CI benchmark thus provides large-scale empirical corroboration, across 18 models and 100 tasks, that the theoretical impossibility results established in this paper are not merely mathematical artifacts but observable phenomena in deployed systems [7, 5].

References

[1] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[2] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[3] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.

[4] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.

[5] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024.

[6] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of FAccT*, pages 610–623, 2021.

[7] J. Chen, X. Xu, H. Wei, C. Chen, and B. Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. *arXiv preprint arXiv:2603.03823*, 2025.

[8] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[10] R. Penrose. *Shadows of the Mind: A Search for the Missing Science of Consciousness.* Oxford University Press, 1994.

[11] N. Bostrom. *Superintelligence: Paths, Dangers, Strategies.* Oxford University Press, 2014.

[12] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[13] A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.